

NetOps MCP Assistant

AI-Assisted Linux Network Configuration and Diagnostic Assistant

CS331 - Computer Networks — Team T018

Team T018

September 11, 2026

Abstract

This project presents an AI-assisted network configuration system that allows users to manage Linux networking through natural-language requests. A lightweight language model accessed through the Groq API interprets the request and produces a structured tool intent, while the Model Context Protocol (MCP) provides a controlled interface to the network-management layer. The system supports firewall configuration using `iptables`, bandwidth shaping using `tc`, Linux kernel parameter profiles using `sysctl`, network diagnostics, empirical benchmarking, and rollback. A declarative policy file validates sensitive operations before execution, and an independent verification layer checks the resulting firewall state, live TCP behavior, and profile configuration. For optimization profiles, the system can additionally perform controlled before-and-after workload measurements using `ping` and `iperf3`. A desktop graphical interface was developed to expose these capabilities without requiring the user to know low-level Linux networking commands.

Keywords: MCP, Linux networking, network automation, `iptables`, `tc`, `sysctl`, verification, benchmarking, natural-language interface

Contents

1	Introduction and Problem Statement	3
1.1	Background	3
1.2	Problem Statement	3
1.3	Objectives	3
2	Approach and Methodology	3
2.1	Natural-Language Intent Interpretation	3
2.2	MCP-Based Execution	3
2.3	Policy-First Execution	4
2.4	Verification-Driven Method	4
3	System Architecture	4
3.1	Architecture Overview	4
3.2	GUI Layer	5
3.3	Assistant and LLM Layer	5
3.4	MCP Client and Server	5
3.5	Policy Layer	6
3.6	Network Operations Layer	6
4	Implementation	6
4.1	Firewall Management	6
4.2	Bandwidth Management	6
4.3	Network Optimization Profiles	6
4.4	Benchmarking	7
4.5	Rollback and Restore	7
4.6	Diagnostics	7
5	Verification and Safety	7
5.1	Firewall Verification	7
5.2	Profile Configuration Verification	8
5.3	Performance Verification	8
5.4	Policy Rejection and Safety	8
6	User Interface Design	8
6.1	UI Goals	8
6.2	Example Conversation Flow	8
7	Results and Evaluation	9
7.1	Firewall Verification Results	9
7.2	Profile Configuration Results	9
7.3	Empirical Benchmark Results	9
7.4	Visual Evidence	9
7.5	Rollback Verification	10
7.6	Limitations	11
8	Challenges Faced	11
8.1	Separating LLM Interpretation from Network Execution	11
8.2	Building Useful Verification	11
8.3	Docker Privilege and Namespace Constraints	11
9	Conclusion	11

1 Introduction and Problem Statement

1.1 Background

Linux networking is powerful but exposes many low-level controls. Firewall rules, traffic shaping, TCP parameters, and diagnostic tools are typically configured through command-line utilities such as `iptables`, `tc`, `sysctl`, `ping`, `ss`, and `iperf3`. This requires the user to know both the intended outcome and the exact syntax needed to achieve it.

In contrast, a user or administrator often thinks in terms of high-level intent, such as “block TCP port 9999”, “limit the interface to 10 Mbps”, or “optimize the connection for gaming”. The project therefore focuses on reducing the gap between high-level intent and low-level Linux network configuration while preserving explicit control over what the system is allowed to execute.

1.2 Problem Statement

The project aims to build a controlled network configuration assistant that translates natural-language requests into safe, structured, and executable network operations while enforcing policy constraints and independently verifying the resulting state.

1.3 Objectives

1. Accept natural-language networking requests without requiring the user to know Linux command syntax.
2. Convert user intent into structured and schema-compatible tool calls.
3. Execute network operations through an MCP-controlled server rather than allowing the language model to invoke shell commands directly.
4. Enforce deterministic security policies before sensitive network changes are executed.
5. Independently verify firewall state, observable network behavior, and supported profile configuration.
6. Provide workload-oriented optimization profiles, empirical benchmarking, rollback, diagnostics, and a graphical interface.

2 Approach and Methodology

2.1 Natural-Language Intent Interpretation

The language model is used as an intent interpreter. It identifies the operation, relevant arguments, and workload profile requested by the user. The model does not directly execute operating-system commands and is not treated as the security authority.

User Request $\longrightarrow$ LLM Interpretation $\longrightarrow$ Structured Tool Intent

For example, a request such as “block TCP port 9999” is converted into a structured firewall operation with action `DROP`, protocol `tcp`, and port `9999`.

2.2 MCP-Based Execution

The assistant sends the structured operation through an MCP client to a FastMCP server using standard input/output streams. The server exposes controlled tools for firewall management, traffic shaping, diagnostics, profiles, benchmarking, and restoration.

Assistant $\longrightarrow$ MCP Client $\xrightarrow{\text{stdio / JSON-RPC}}$ FastMCP Server $\longrightarrow$ Network Tools

This separation prevents the language-model layer from becoming the execution authority and keeps the actual network changes inside the server-side tool boundary.

2.3 Policy-First Execution

Before a system command is invoked, the requested action is checked against the declarative rules in `rules/policies.yaml`. The current policy defines protected ports, protected interfaces, allowed firewall actions and protocols, and bandwidth limits.

$$\text{Request} \rightarrow \text{Policy Validation} \rightarrow \begin{cases} \text{Reject} & \text{if not allowed} \\ \text{Execute} & \text{if allowed} \end{cases}$$

For example, ports 22, 53, and 5000 are protected. A request to drop traffic on one of these ports is rejected before the network command is executed.

2.4 Verification-Driven Method

The project follows the principle:

Validate $\rightarrow$ Execute $\rightarrow$ Verify

A successful process exit code alone is not treated as proof that the intended network state exists. Instead, the verifier inspects the active network state and, where applicable, performs an independent behavioral test.

For optimization profiles, the evaluation cycle is:

Measure Baseline $\rightarrow$ Apply Profile $\rightarrow$ Measure Workload $\rightarrow$ Compare

This distinction allows the report to separate two claims: whether a configuration was applied successfully, and whether a measurable performance change was observed.

3 System Architecture

3.1 Architecture Overview

The system is organized into a user interface layer, an orchestration layer, an intent-interpretation layer, an MCP execution layer, a Linux network-operation layer, and a verification layer.

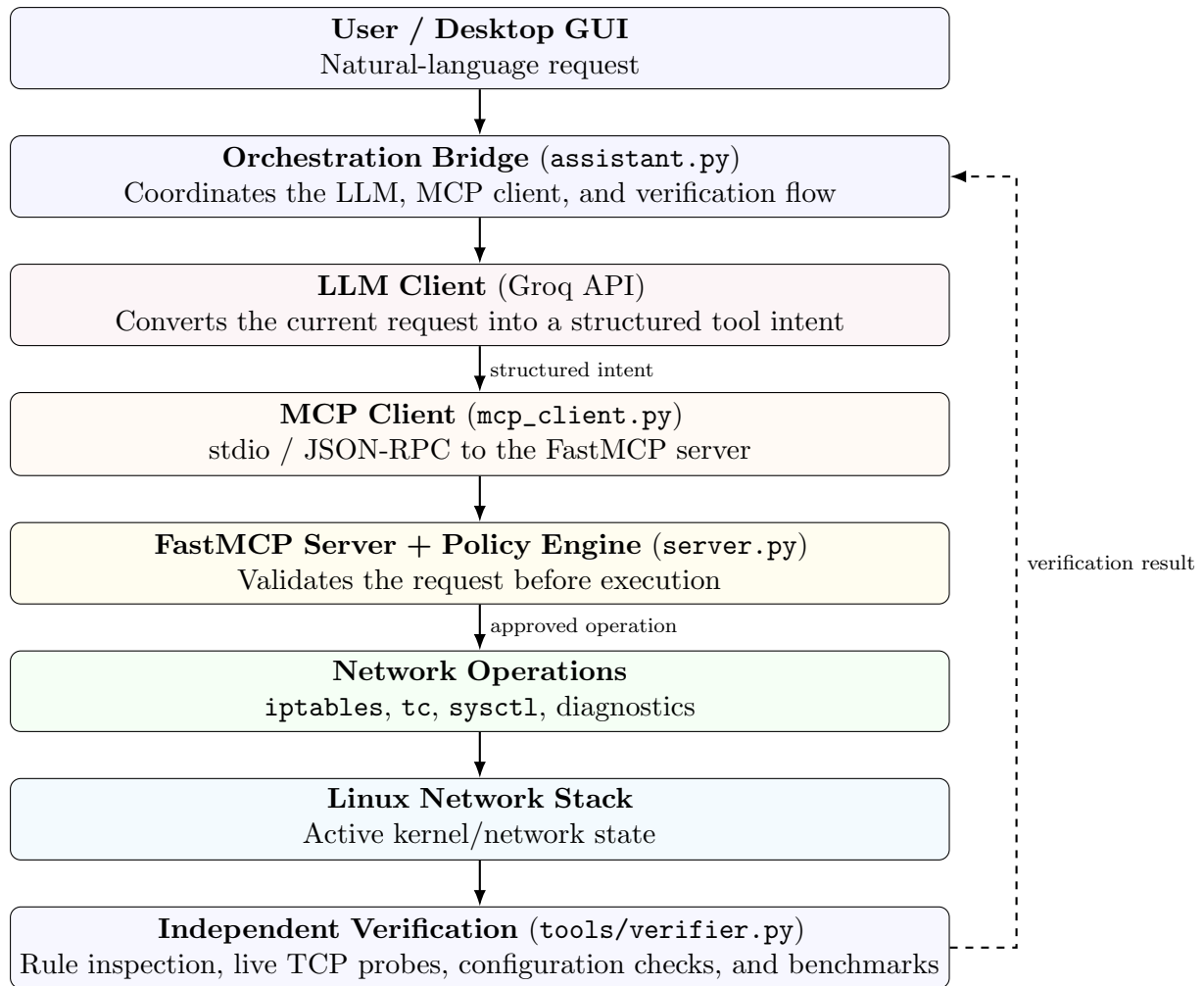


Figure 1: System architecture and request-to-verification flow.

3.2 GUI Layer

The interface is implemented with HTML, CSS, and JavaScript and can be hosted as a desktop application through `pywebview` or used in web mode. The interface is designed around a normal conversation flow, with the user on the right and assistant responses on the left. During an operation, the GUI displays processing stages, the verification result, and relevant execution evidence.

3.3 Assistant and LLM Layer

The `NetOpsBridge` in `assistant.py` coordinates the main request lifecycle. It sends the current user request to the LLM client, receives a structured tool call, invokes the requested MCP tool, and returns the result to the interface. The LLM therefore participates in interpretation, not authorization or direct operating-system execution.

3.4 MCP Client and Server

The MCP client in `mcp_client.py` communicates with the FastMCP server over stdio and asynchronous JSON-RPC. The server exposes network operations as explicit tools. This creates a clear boundary between conversational interpretation and privileged network control.

3.5 Policy Layer

The policy engine reads `rules/policies.yaml`. Important controls include:

- protected ports: 22 (SSH), 53 (DNS), and 5000 (application/UI service);
- protected interfaces: `eth0` and `lo`;
- valid port range: 1 to 65535;
- allowed firewall actions: `ACCEPT`, `DROP`, and `REJECT`;
- allowed protocols: `TCP` and `UDP`;
- bandwidth limits bounded by the configured minimum and maximum values.

3.6 Network Operations Layer

The main operating-system mappings are shown in Table 1.

Function	Linux mechanism
Firewall management	<code>iptables</code> filtering rules
Bandwidth shaping	Linux Traffic Control (<code>tc</code>) using a supported queueing discipline
Kernel parameters	<code>sysctl / /proc/sys/net/</code>
Port reachability	TCP socket connection probe
Diagnostics	<code>ping</code> , <code>ss</code> , and <code>iperf3</code>

Table 1: Mapping of project operations to Linux networking mechanisms.

4 Implementation

4.1 Firewall Management

A natural-language firewall request is converted to the `configure_firewall` operation. For example:

```
User: Block TCP port 9999
Intent: configure_firewall(action=DROP, protocol=tcp, port=9999)
Policy: approved
Command: iptables -A INPUT -p tcp --dport 9999 -j DROP
```

The implementation supports `ACCEPT`, `DROP`, and `REJECT`. Policy validation takes place before the command is executed.

4.2 Bandwidth Management

Bandwidth control uses Linux Traffic Control. A representative operation for an allowed interface is:

```
tc qdisc replace dev eth1 root tbf rate 10mbit burst 32kbit latency 50ms
```

The system prevents bandwidth restrictions from being applied to the protected interfaces `eth0` and `lo`.

4.3 Network Optimization Profiles

The project provides three workload-oriented profiles. Their purpose is to provide a repeatable collection of Linux networking settings rather than requiring the user to configure each parameter manually.

Gaming. The Gaming profile targets low-latency workloads and uses supported TCP and queue-management settings intended to reduce queuing delay and maintain responsive connections. The profile can be evaluated using latency, jitter, packet loss, and connection-time measurements.

Streaming. The Streaming profile targets receiver-side TCP workloads. It adjusts supported receive-window and related TCP settings intended for sustained data reception.

Broadcasting. The Broadcasting profile targets sender-side traffic. It adjusts supported send-side parameters and TCP pacing/queue settings intended for sustained outbound workloads.

Because some network parameters are controlled by the host namespace in a containerized environment, a profile can report settings as applied or skipped depending on the capabilities exposed to the container.

4.4 Benchmarking

The `run_profile_benchmark` operation follows a controlled before-and-after process. The system first records baseline measurements, applies the target profile through MCP, repeats the workload, and reports the resulting deltas. `ping` provides latency, jitter, and packet-loss measurements, while `iperf3` provides throughput-oriented workload measurements.

The benchmark is therefore used as behavioral evidence, not as a guarantee that every profile improves every metric in every environment.

4.5 Rollback and Restore

Before supported kernel parameters are modified, their current values are saved to a checkpoint file. The restore operation uses those saved values to return the system toward the previous configuration. This allows failed or unsuitable profile changes to be reverted without requiring the user to manually reconstruct the earlier settings.

4.6 Diagnostics

The system includes tools for listing listening ports, testing port connectivity, checking bandwidth, and running general network diagnostics. These tools are also useful for verifying the effect of a firewall rule or confirming the state of the network before an optimization experiment.

5 Verification and Safety

5.1 Firewall Verification

Firewall verification uses two complementary checks.

1. **Rule-state verification:** the verifier inspects the active firewall table and checks whether the requested rule is present.
2. **Live behavior verification:** a TCP socket probe tests the observable behavior of the target port.

For a `DROP` rule, a timeout or blocked connection is consistent with the expected behavior. For `REJECT`, an actively refused connection is consistent with the rule. For `ACCEPT`, a connection can be established when a listening service exists; otherwise a refusal can indicate that the service is closed rather than that the firewall blocked the connection.

Thus, firewall verification is based on both ****what the active rule table contains**** and ****what happens to an actual connection attempt****.

5.2 Profile Configuration Verification

After a profile is applied, supported `sysctl` values are read back from the live system and compared with the intended profile values. This establishes whether the requested configuration was actually written in the current network namespace.

5.3 Performance Verification

Performance verification is distinct from configuration verification. A successful parameter update does not by itself prove that an application workload improved. The benchmark therefore compares measurements before and after profile application.

The measured metrics depend on the workload:

- Gaming: latency, jitter, packet loss, and TCP connection timing.
- Streaming: receiver-side TCP throughput, retransmissions, and transferred data.
- Broadcasting: sender-side TCP throughput, retransmissions, and transferred data.

5.4 Policy Rejection and Safety

Protected ports are rejected by the deterministic policy layer before any network command is executed. Command arguments are passed as structured subprocess arguments rather than through shell interpolation, reducing command-injection risk. The system also records checkpoint information before supported profile mutations so that restoration remains possible after an error.

6 User Interface Design

6.1 UI Goals

The UI is designed to hide unnecessary Linux complexity while keeping important operational evidence visible. The main goals are:

- conversational interaction rather than command-line syntax;
- visible processing stages during network operations;
- clear success, rejection, and verification states;
- expandable execution evidence when a command actually runs;
- compact benchmark cards for before-and-after measurements.

6.2 Example Conversation Flow

User: Block TCP port 9999

Assistant:

```
Understanding the request ...
Checking the applicable policy ...
Applying the network change ...
Verifying the firewall rule ...
Verifying live connectivity ...
```

Result:

```
TCP port 9999 is blocked and verified.
```

The interface is intended to expose the verification result in user-friendly language while retaining technical details for debugging and demonstration.

7 Results and Evaluation

This section evaluates the system from three perspectives: firewall correctness, profile configuration, and workload benchmarking. The main evidence is intentionally separated into configuration evidence and performance evidence so that successful execution is not confused with guaranteed performance improvement.

7.1 Firewall Verification Results

Table 2 summarizes representative firewall checks.

Operation	Policy	Rule Check	Live Probe	Status
DROP tcp:9999	PASS	PASS	PASS (blocked/refused)	VERIFIED
REJECT tcp:9999	PASS	PASS	PASS (refused)	VERIFIED
ACCEPT tcp:9999	PASS	PASS	PASS (connected in test setup)	VERIFIED
DROP tcp:22	REJECTED	Not executed	Not tested	BLOCKED
DROP tcp:53	REJECTED	Not executed	Not tested	BLOCKED

Table 2: Representative firewall verification matrix.

7.2 Profile Configuration Results

The containerized test environment applied supported parameters while reporting host-constrained settings as skipped. The observed configuration status is summarized below.

Profile	Applied	Skipped / constrained	Configuration status
Gaming	5	2	Applied and verified
Streaming	6	2	Applied and verified
Broadcasting	6	3	Applied and verified

Table 3: Profile parameter application in the container environment.

7.3 Empirical Benchmark Results

The project includes measured before-and-after benchmarking. A representative Gaming benchmark showed a reduction in measured jitter from 2.21 ms to 1.40 ms, corresponding to approximately a 36.6% reduction. This result is useful as an example of a positive measured effect; it should be interpreted as an observation for the tested environment rather than a universal guarantee.

Gaming metric	Before	After	Observed change
Jitter	2.21 ms	1.40 ms	-36.6%

Table 4: Representative positive Gaming-profile benchmark result.

7.4 Visual Evidence

The following figures provide a direct visual trace of two representative system capabilities: firewall execution at the Linux level and a measured optimization result for the Gaming profile.

Together, they complement the tabulated verification results by showing both the requested operation and the observed system state.

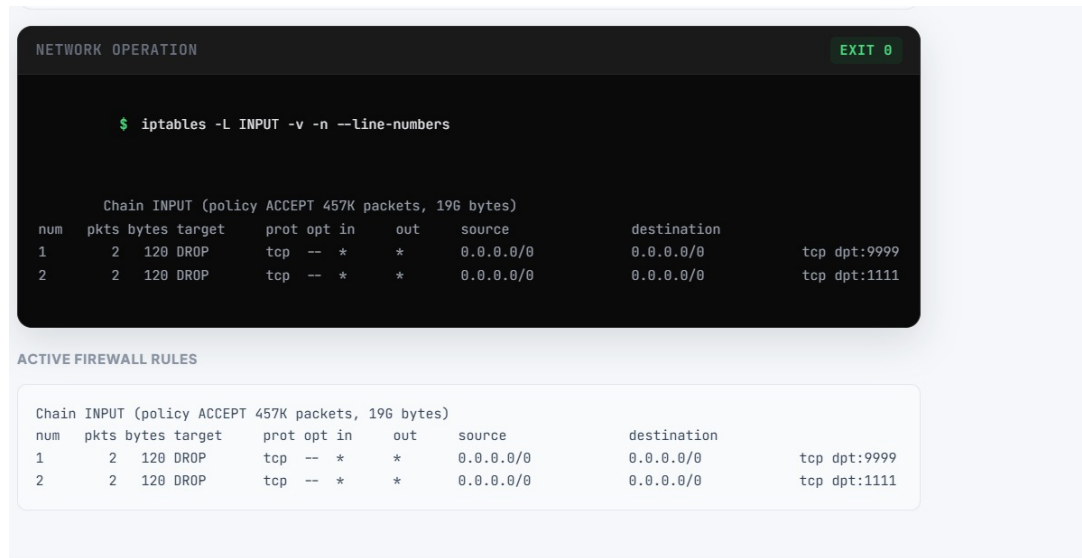


Figure 2: Graphical interface showing the active firewall rules, including the TCP port 9999 rule.

```
root@d7b4b10d83e5:/app# iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy ACCEPT 457K packets, 196 bytes)
num  pkts bytes target    prot opt in     out     source    destination
1    2    120 DROP      tcp  --  *      *        0.0.0.0/0  0.0.0.0/0          tcp dpt:9999
2    2    120 DROP      tcp  --  *      *        0.0.0.0/0  0.0.0.0/0          tcp dpt:1111
root@d7b4b10d83e5:/app#
```

Figure 3: Linux-level firewall state inspected with `iptables`; the active table contains DROP rules for TCP ports 9999 and 1111.

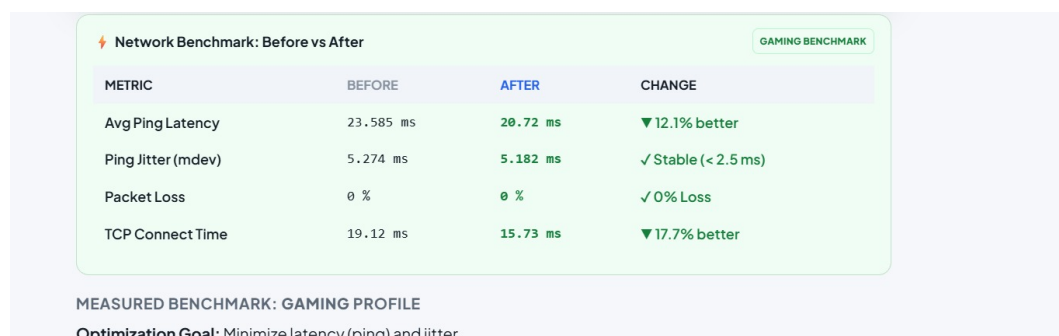


Figure 4: Gaming-profile benchmark showing before-and-after latency, jitter, packet loss, and TCP connection-time measurements.

Figure 2 shows the same firewall state exposed through the application interface, while Figure 3 shows the corresponding Linux firewall table directly. Figure 4 provides the profile-level performance evidence used for the positive Gaming result discussed above.

7.5 Rollback Verification

Rollback was also tested by observing a parameter transition from the baseline value to a profile value and then restoring the previous value. A representative state sequence is shown below.

Parameter	Baseline	Tuned	Restored	Result
tcp_congestion_control	cubic	bbr	cubic	Successfully reverted
tcp_fin_timeout	60	15	60	Successfully reverted
tcp_fastopen	1	3	1	Successfully reverted

Table 5: Representative rollback state transition.

7.6 Limitations

1. **Container versus host isolation:** some Linux networking parameters are controlled by the host network namespace and cannot always be changed from the container environment.
2. **Workload specificity:** profile effects depend on the workload and network bottleneck. A configuration can be applied successfully without improving every metric.
3. **Service dependency for firewall testing:** an ACCEPT rule is easier to demonstrate when a service is listening on the target port; otherwise a refusal can simply indicate that the application is closed.
4. **Benchmark environment:** very high local or virtualized throughput measurements should be interpreted as properties of the test path, not as Internet throughput guarantees.

8 Challenges Faced

8.1 Separating LLM Interpretation from Network Execution

A central design challenge was establishing a strict boundary between natural-language interpretation and privileged network operations. The resulting architecture treats the LLM as an interpreter while the MCP server and policy layer control execution.

8.2 Building Useful Verification

A simple connectivity test such as ICMP ping is not sufficient to prove the correctness of a firewall rule. The verifier was therefore expanded to combine active TCP probing with inspection of the firewall rule table.

8.3 Docker Privilege and Namespace Constraints

Operations such as `iptables` and `tc` require appropriate Linux capabilities. The container environment also exposes only a subset of host-level networking controls, so the implementation had to distinguish between supported settings and settings that were unavailable or read-only in the current namespace.

9 Conclusion

The NetOps MCP Assistant demonstrates a practical approach to natural-language Linux network administration without making the language model the authority over network safety. User requests are converted into structured tool calls, passed through an MCP client to a FastMCP server, checked against deterministic policies, and executed using Linux networking tools. The resulting state is then verified using firewall inspection, live socket behavior, or profile configuration checks, with optional before-and-after workload benchmarking for optimization profiles.

The project therefore combines usability with explicit execution boundaries, policy enforcement, verification, and rollback. The measured results also illustrate an important engineering point: successful configuration and improved performance are separate claims. The system can verify that a profile was applied and can measure its effect, while the actual performance outcome remains dependent on the workload and test environment.

References

- [1] Anthropic, *Model Context Protocol Specification*, 2024.
<https://modelcontextprotocol.io/>
- [2] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, “BBR: Congestion-Based Congestion Control,” *ACM Queue*, vol. 14, no. 5, 2016.
- [3] K. Nichols and V. Jacobson, “Controlling Queue Delay (CoDel),” *Communications of the ACM*, vol. 55, no. 7, 2012.
- [4] S. Ha, I. Rhee, and L. Xu, “CUBIC: A New TCP-Friendly High-Speed TCP Variant,” *ACM SIGOPS Operating Systems Review*, vol. 42, no. 5, 2008.
- [5] V. Jacobson, R. Braden, and D. Borman, “TCP Extensions for High Performance,” RFC 1323, 1992.